

PROJECT APEX

Per-Asset Feature Panel
Evaluation Report

Panel: `Ticker_Data/panels/fold_1/feature_panel.npz` | Shape: `[5,577 × 110 × 26]`
Date range: 2004-01-02 → 2026-03-03 | Generated: March 09, 2026

PASS	WARN	FAIL	TOTAL CHECKS
			28

Automated suite: 28 checks across shape, clipping, causality, CS distributions, raw data, and cross-fold identity. Additional deep inspection revealed **4 further critical issues** (Issues 2–5 below) not surfaced by the automated checks.

ISSUES OVERVIEW

#	Issue	Severity	Feature(s)	Scope
1	`adj_close` == `close` — duplicate slot	CRITICAL	adj_close (F=3)	All T × all active assets
2	`ret_rank_4w` in [0,1], not N(0,1) — scale mismatch	CRITICAL	ret_rank_4w (F=18)	All T × all active assets
3	`momentum_sector_residual` std = 0.085 — near-zero signal	CRITICAL	momentum_sector_residual (F=25)	All T × all active assets
4	Tradability mask ignores null open/close — §2.4.1 violation	CRITICAL	mask_panel	119–771 data-days
5	Raw bars: 771 null opens, 119 null close/volume	MODERATE	All features for ~200 securities	890 data-days
6	volume=0 rows not gated as non-tradeable	MODERATE	volume_z_4w, mask_panel	418 rows, 15 securities
7	`open` redundant with `close`; price levels biased +0.40σ	DESIGN	open (F=0)	All T
8	`vol_52w` min_periods=260 — new entrants zero for ~1 year	DESIGN	vol_52w (F=10)	~2.9% of active slots

FEATURE STATISTICS (active slots only)

Statistics computed across all active (mask=1) asset-day slots in fold_1. Flagged columns: **[HIGH-ZERO]** >20% zeros · **[LOW-VAR]** std < 0.1 · **[BIASED]** |mean| > 0.3.

Feature	Mean	Std	Min	Max	% Zero	% @Clip	Flag
open	+0.402	1.370	-4.000	4.000	3.9%	0.14%	BIASED
close	+0.402	1.370	-4.000	4.000	3.9%	0.14%	BIASED
volume	+0.007	0.915	-2.876	4.000	3.9%	1.03%	
adj_close	+0.402	1.370	-4.000	4.000	3.9%	0.14%	BIASED / DUPE
log_ret	-0.006	0.951	-4.000	4.000	3.9%	0.60%	
ret_1w	-0.015	0.996	-4.000	4.000	3.9%	0.43%	
ret_4w	-0.035	1.050	-4.000	4.000	3.9%	0.22%	
ret_12w	-0.067	1.158	-4.000	4.000	3.9%	0.13%	

vol_1w	-0.004	1.009	-2.719	4.000	3.9%	1.08%	
vol_4w	+0.012	1.147	-3.663	4.000	3.9%	0.97%	
vol_52w	+0.031	1.564	-4.000	4.000	6.8%	0.84%	HIGH-ZERO
volume_z_4w	+0.001	0.974	-3.122	4.000	3.9%	0.05%	
beta_26w_mkt	-0.022	1.388	-4.000	4.000	3.9%	0.28%	
rel_strength_4w	-0.031	1.055	-4.000	4.000	3.9%	0.17%	
vol_ratio_1w_4w	-0.001	0.989	-3.011	3.809	3.9%	0.00%	
RSI_14	-0.016	1.039	-4.000	4.000	3.9%	0.00%	
bollinger_percent_b	-0.009	0.999	-4.000	3.855	3.9%	0.00%	
bollinger_bandwidth	+0.007	1.077	-3.230	4.000	3.9%	0.89%	
ret_rank_4w	+0.485	0.299	0.000	1.000	3.9%	0.00%	SCALE MISMATCH
ret_z_4w	+0.000	0.968	-4.000	4.000	3.9%	0.36%	
ret_z_12w	-0.000	0.972	-4.000	4.000	4.0%	0.21%	
vol_z_4w	-0.007	0.945	-3.650	4.000	3.9%	0.62%	
volume_z_cs_4w	-0.002	0.971	-3.549	4.000	3.9%	0.38%	
ret_z_4w_sector	+0.000	0.969	-4.000	4.000	5.8%	0.06%	
vol_z_4w_sector	-0.001	0.967	-3.212	4.000	5.8%	0.16%	
momentum_sector_residual	-0.000	0.085	-2.507	4.000	5.8%	0.00%	LOW-VAR

CRITICAL ISSUES (4) — Fix Before Remaking Panels

CRITICAL

Issue 1: adj_close == close — Duplicate Feature Slot

File: *features/per_asset_features.py* line 159

daily_bars.parquet contains only open, close, and volume. There is no separate adj_close column — the parquet's close column *already* is the total-return adjusted close. The implementation sets `adj_close = close`, producing two byte-for-byte identical feature channels at indices F=1 and F=3.

Confirmed: **diff_max = 0.00e+00** across all $5,577 \times 110$ slots. The model wastes one of its 18 TS feature slots on zero marginal information. The TCN will learn these channels are identical, permanently wasting that representational capacity.

```
# per_asset_features.py:159 – current (WRONG) 'adj_close': close, # canonical: close
already adj. # Fix: replace adj_close with overnight_gap_return 'overnight_gap':
np.log(open_ / close.shift(1)) # also feeds $5.4 cost model
```

Recommended Fix:

Replace `adj_close` with `overnight_gap_return = log(open_t / close_{t-1})`. This is: (a) a stationary return, not a trending level; (b) distinct from `close`; and (c) directly useful to the \$5.4 gap volatility cost model. Update the feature name in `TS_FEATURE_NAMES`, the panel builder, and `master_config.yaml`.

CRITICAL

Issue 2: ret_rank_4w Stays in [0, 1] — Scale Mismatch with All Other Features

File: *features/cross_sectional_features.py* lines 50–58 & *feature_panel.py* lines 162–168

`_cs_rank()` correctly returns percentile ranks in $[0, 1]$. However, when `_pack_day_into_slot()` assembles the panel, CS features are read directly from `cs_feat[sid]` without passing through `CausalPerAssetNormalizer`. All 25 other features span $[-4.0, +4.0]$. `ret_rank_4w` is permanently confined to $[0.0, 1.0]$ — occupying only 12.5% of the feature scale.

Measured: **mean = +0.485, std = 0.299, max = 1.000**. The model can never see this feature go above 1.0 while all neighbours saturate at 4.0. This systematically compresses the signal relative to the initialisation and training assumptions of the TCN.

```
# cross_sectional_features.py – current (WRONG) ret_rank_4w = _cs_rank(ret_4w_vals) #
→ [0, 1] # Fix option A: centre the rank ret_rank_4w = 2.0 * _cs_rank(ret_4w_vals) -
1.0 # → [-1, +1], mean=0 # Fix option B: z-score the rank (uniform → approx
N(0,0.289)) ret_rank_4w = _cs_zscore(_cs_rank(ret_4w_vals)) # → mean=0, std≈1
```

Recommended Fix:

Apply `2 * rank - 1` (Option A) to map to $[-1, +1]$ centred at 0, or apply `_cs_zscore(rank)` (Option B) to put it on the same $N(0,1)$ scale as `ret_z_4w`. Option A is simpler and preserves the bounded monotonic property.

CRITICAL

Issue 3: momentum_sector_residual Has Near-Zero Variance (std = 0.085)

File: *features/cross_sectional_features.py* lines 137–142

Computed as $\text{ret_4w} - \text{sector_mean}(\text{ret_4w})$ in raw log-return units. Sector members are highly correlated so intra-sector residuals are tiny ($\sim \pm 0.01$ – 0.05 per 4 weeks). The feature is then per-asset z-scored by `CausalPerAssetNormalizer`, but the rolling std is also tiny, so the normalizer amplifies noise rather than signal.

Measured: **std = 0.085** vs $\text{std} \approx 0.97$ for every other CS feature. This feature is **11× lower variance** than `ret_z_4w_sector`, which encodes the same information but is properly cross-sectionally z-scored. The feature is semantically redundant and numerically near-dead.

```
# cross_sectional_features.py – current (WRONG) momentum_sector_residual =
ret_4w_vals - sector_mean_ret # raw return units # Fix: divide by sector std →
properly scaled residual sector_std_ret = np.full_like(ret_4w_vals, 1.0) for sector
in np.unique(sector_codes): mask = sector_codes == sector finite =
ret_4w_vals[mask][np.isfinite(ret_4w_vals[mask])] if len(finite) >= 2:
sector_std_ret[mask] = max(float(finite.std()), EPS) momentum_sector_residual =
(ret_4w_vals - sector_mean_ret) / sector_std_ret
```

Recommended Fix:

Divide the raw residual by the sector's own std. This yields a standardised residual with $\text{std} \approx 1.0$ across the full active universe, placing it on the same scale as all other CS features and making it genuinely distinct from `ret_z_4w_sector` (which uses the cross-sector as its zero baseline, not per-sector standardisation).

CRITICAL

Issue 4: Tradability Mask Ignores Null open/close — §2.4.1 Violation

File: `features/feature_panel.py _pack_day_into_slot()` & `feature_panel.py _build_asof_membership()`

The mask is set to 1.0 for all NDX members regardless of whether the asset has valid price data on that date. Per §2.4.1: an asset is tradeable only if it has a valid non-NaN adjusted close on `d_t` and a valid open on `d_{t+1}`. When `close=NaN`, the normaliser forward-fills to 0.0, producing a seemingly normal feature vector for an asset that cannot be priced or executed.

Scope: 119 rows with `close=NaN` across 112 securities, 771 rows with `open=NaN` across 195 securities, 418 rows with `volume=0` across 15 securities. The environment would attempt to trade these assets and apply the \$5.4 cost model to a price that does not exist.

```
# feature_panel.py – add validity gate in _pack_day_into_slot() for sid in
active_sids_sorted: if slot >= K_max: break # NEW: check valid close price exists if
sid not in ts_norm or date not in ts_norm[sid].index: slot += 1 continue raw_close =
ts_raw[sid].loc[date].get('close', np.nan) if not np.isfinite(raw_close) or raw_close
<= 0: continue # keep mask=0, skip slot increment # ... rest of packing logic
mask_day[slot] = 1.0
```

Recommended Fix:

Pass the raw (pre-normalization) TS features dict `ts_raw` into `_pack_day_into_slot()` alongside `ts_norm`. Before setting `mask=1.0`, check that `close > 0` in `ts_raw`. Additionally, the data pipeline should flag `volume=0` rows as non-tradeable in the bars before features are computed.

MODERATE ISSUES (2) — Data Quality

MODERATE

Issue 5: Raw Bars Contain 771 Null Opens and 119 Null Close / Volume Rows

File: `Ticker_Data/daily_bars.parquet`

Null coverage in `daily_bars.parquet`:

Field	Null Rows	Securities Affected	Notes
open	771	195	652 of these have valid close — execution price gap
close	119	112	Full data gap (close+open+volume all null)
volume	119	112	Co-occurs with null close

Notable cluster: Large batch of null open rows on 2005-06-21 and 2005-06-23 across many securities — likely a data provider outage or import error, not actual market halts. **Fix:** Add a data validation pass at ingestion to detect, log, and either forward-fill (using prior day close as proxy open) or flag as non-tradeable.

MODERATE

Issue 6: volume=0 Rows Are Not Gated as Non-Tradeable

File: `Ticker_Data/daily_bars.parquet` + `features/feature_panel.py`

418 rows across **15 securities** have `volume=0` (and simultaneously `open=NaN`), spanning **2004-01-22 to 2024-11-20**. One security has 5 consecutive zero-volume days (2022-11-28 – 2022-12-28). Per §2.4.1, zero volume implies the asset is non-tradeable, but the panel mask does not reflect this.

`volume_z_4w` for these rows resolves as $(0 - \text{rolling_mean}) / \text{rolling_std}$, producing a large negative z-score (or 0 if the entire 20-day window is also zero). This sends a misleading 'below-average volume' signal rather than a 'non-tradeable' signal.

Recommended Fix:

Gate `mask=0` whenever `volume <= 0` in the raw bars (same fix path as Issue 4). Also add a `volume_valid` boolean column to `daily_bars` during ingestion.

DESIGN CHOICES TO REVIEW (2)

DESIGN

Issue 7: open Redundant with close; Price Level Z-Scores Carry +0.4 Bull-Market Bias

File: `features/per_asset_features.py` lines 107–108, 154–158

Both `open` and `close` are price *levels* z-scored over a 260-day rolling window. In a predominantly bullish 2004–2026 sample, prices tend to be above their own 260-day mean, producing a persistent **+0.402 σ mean bias** for all three price-level features (`open`, `close`, `adj_close`). A zero-mean signal is an implicit assumption for most normalisation-sensitive network initialisations.

Additionally, `open` and `close` are tracking the same price series — they differ only by the intraday return (typically <0.5%). The two features convey nearly identical information, wasting representational capacity. The bible §3.1 lists them as separate features but does not mandate they be raw levels rather than returns.

Recommended Fix:

If `adj_close` is replaced with `overnight_gap` (Issue 1 fix), consider also replacing `open` with `intraday_return = log(close / open)`, which is stationary, zero-mean, and distinct from `log_ret` (close-to-close). This would leave `close` as the only price-level feature (a useful anchor) while making the other price-related slots return-based.

DESIGN

Issue 8: `vol_52w` Uses `min_periods=260` — New NDX Entrants Have Zero Volatility for ~1 Year

File: `features/per_asset_features.py` line 122

`vol_52w` is computed with `min_periods=DAYS_52W=260`. Any security with fewer than 260 days of price history produces NaN, which the normaliser fills with 0.0 — signalling 'perfectly average volatility' to the model for approximately the first trading year after NDX entry. This is precisely when accurate risk estimation matters most.

Measured: `vol_52w` zero rate = **6.8%** vs 3.9% for all other features. The extra 2.9% ($\approx 3,800$ asset-days) represents new entrants during their first year.

```
# per_asset_features.py line 122 – current vol_52w = log_ret.rolling(DAYS_52W,
min_periods=DAYS_52W).std() * ANNUALIZE # Fix: allow early estimation from partial
window vol_52w = log_ret.rolling(DAYS_52W, min_periods=DAYS_4W).std() * ANNUALIZE
```

Recommended Fix:

Change `min_periods=DAYS_52W` to `min_periods=DAYS_4W` (20). The estimate is noisier early on but is directionally correct and non-zero, which is far better than a silent zero. The 260-day window still governs the smoothing horizon once sufficient history is available.

RECOMMENDED REMEDIATION ORDER

Issues 1–4 affect every single active asset on every date — they are foundational enough to make any model trained on the current panels unreliable. Complete all four critical fixes, then rebuild all 8 fold panels using `scripts/build_all_panels.py`.

Step	Action	File(s)	Priority
1	Replace <code>adj_close</code> with <code>overnight_gap_return</code>	<code>per_asset_features.py</code> <code>feature_panel.py</code> <code>master_config.yaml</code>	CRITICAL
2	Centre <code>ret_rank_4w</code> : apply <code>2×rank-1</code> transformation	<code>cross_sectional_features.py</code>	CRITICAL
3	Standardise <code>momentum_sector_residual</code> by sector std	<code>cross_sectional_features.py</code>	CRITICAL
4	Add null/zero-volume price gate to tradability mask	<code>feature_panel.py</code> <code>dailyBars</code> ingestion	CRITICAL
5	Change <code>vol_52w</code> <code>min_periods</code> to <code>DAYS_4W</code> (20)	<code>per_asset_features.py</code>	DESIGN
6	Audit and clean null/zero-volume rows in <code>dailyBars.parquet</code>	<code>data_pipeline</code> / <code>update_parquets.py</code>	MODERATE
7	Rebuild all 8 fold panels	<code>scripts/build_all_panels.py</code>	MODERATE
8	Re-run full evaluation suite to confirm all issues resolved	<code>scripts/evaluate_per_asset_panels.py</code>	MODERATE

Automated Checks: PASS / WARN / FAIL Summary

- ✓ Shape & Dimension (F=26, K=110, D_global=20)
- ✓ Clipping enforcement — all active slots within ± 4.0
- ✓ Raw price leakage — open/close/volume are z-scored, not raw
- ✗ `adj_close == close` — identical duplicate slot ← Issue 1
- ✓ No NaN anywhere in `x_panel`
- ✓ All padded (`mask=0`) slots are exactly 0.0
- ✓ Zero-active days only during 2004 warm-up (last: 2004-06-18)
- ✓ Active count never exceeds `K_max=110`
- ✓ `ret_z_4w` / `ret_z_12w` / `vol_z_4w` distributions $\sim N(0,1)$
- ✓ `vol_1w`–`vol_4w` correlation = 0.61 (>0.6 threshold)
- ✓ `vol_4w`–`vol_52w` correlation = 0.37 (below 0.4, acceptable for different windows)
- ✓ Bollinger %B and bandwidth z-score distributions OK

- [✓] beta_26w_mkt: 0.73% near clip — no numerical instability
- [✓] Features change day-to-day — no stuck/cached values
- [✓] x_panel identical across fold_1 and fold_3 (causal norm is fold-independent)
- [✓] g_panel differs across folds (fold-specific fixed-scale normalization)
- [!] ret_rank_4w mean = 0.485 — expected near 0 ← Issue 2
- [!] Raw bars: 771 null opens, 119 null close ← Issues 4–6
- [✓] No duplicate (date, security_id) in daily_bars
- [✓] No zero/negative close prices

Report generated March 09, 2026 at 10:38 — Project Apex Engineering — CONFIDENTIAL